資料視覺化
Data Visualization

Week 11: Text

Github and Deploying Streamlit

What is git?

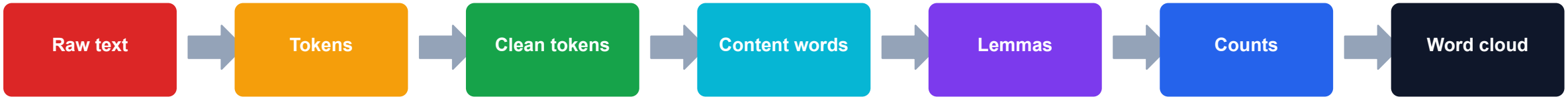
- Git is the tool that tracks changes in your code.
- GitHub is the online platform where Git repositories are stored, shared, reviewed, and collaborated on.
- Basic workflow (for a solo project):
  - Make changes locally
  - Commit changes with Git
  - Push the changes to GitHub
- There is an app: https://desktop.github.com/download/

Natural Language Processing

Some key ideas

- Natural Language Processing (NLP) is basically counting words
- Bag-of-words approach, aka. word order doesn't matter:
  - I love my family and eating food
  - I love food and eating my family
- We (as data scientists) moved on, but it is useful sometimes
- Everyone loves a word cloud (we are doing one today),
  even though it is one of the worst visualizations ever

NLP pipeline



Tokenization

- Splitting sentence into individual words
- We call that “tokens”
- In some cases, tokens are not always words (it can be part of a word)

```
import nltk, re

def tokenize(text):
    try:
        return nltk.word_tokenize(text)
    except LookupError:
        return re.findall(r"[A-Za-z]+(?:'[A-Za-z]+)?", text)

raw_tokens = tokenize(text)
raw_tokens[:15]
```

Normalise tokens

- “Language” and “language” are two different words (tokens)
- We avoid that by converting everything to lowercase
- In most cases we also remove numbers and punctuation

```
clean_tokens = [
    token.lower()
    for token in raw_tokens
    if token.isalpha()
]

clean_tokens[:20]
```

Remove stopwords

- In all human languages, the most frequent words are meaningless
- It will dominate the analysis without telling us anything
- We remove them

```
from nltk.corpus import stopwords

stop_words = set(stopwords.words("english"))
stop_words |= {"nlp", "nltk"}  # domain-specific stopwords

content_words = [w for w in clean_tokens if w not in stop_words]
content_words[:20]
```

Stemming vs. lemmatization

- We what to combine:
  - dogs
  - dog
  - doggies
  - doggo
- Stemming:
  Rule-based chopping.
- Lemmatization:
  Dictionary-based normalization.

```
stemmer = nltk.PorterStemmer()
stemmed = [stemmer.stem(w) for w in content_words]

lemmatizer = nltk.WordNetLemmatizer()
lemmas = [lemmatizer.lemmatize(w) for w in content_words]
```

Count word frequencies

- In most case, we count
- Basic assumption:
  the more times the word appears,
  the more important the word is
- If dog appeared twice and cat appeared once, we think dog is more important
- Remember word order doesn't matter

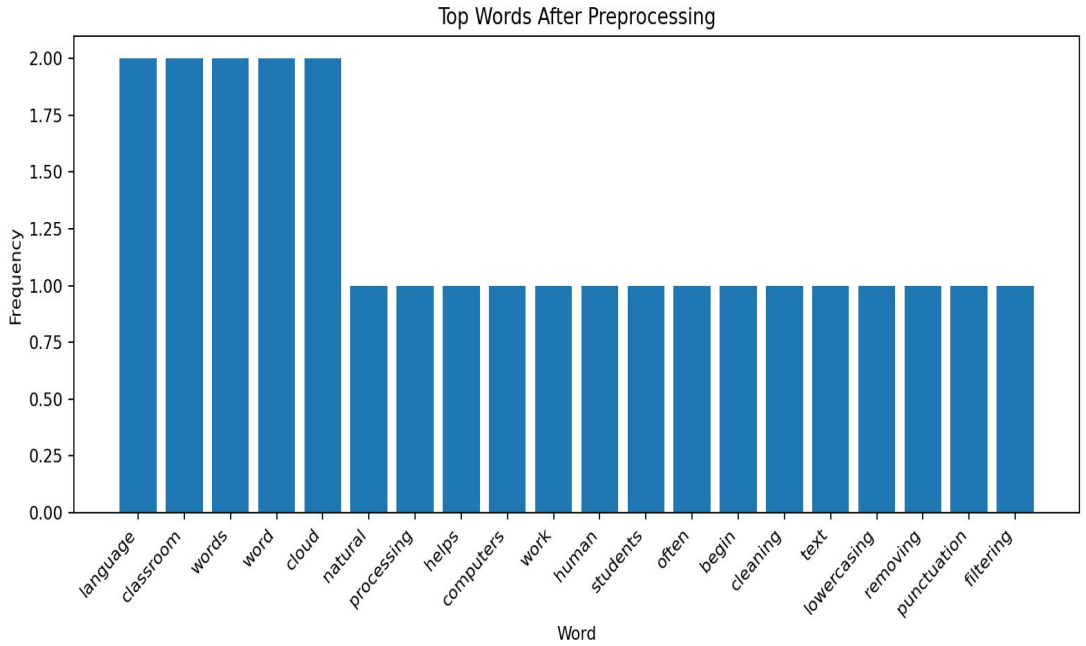
```
from collections import Counter
import pandas as pd

counts = Counter(lemmas)
freq_df = pd.DataFrame(
    counts.most_common(20),
    columns=["word", "count"]
)
freq_df.head()
```

Visualize top words

- Simplest way to visualise word frequencies is a good old bar chart

```
plt.figure(figsize=(10, 5))
plt.bar(freq_df["word"], freq_df["count"])
plt.title("Top Words After Preprocessing")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

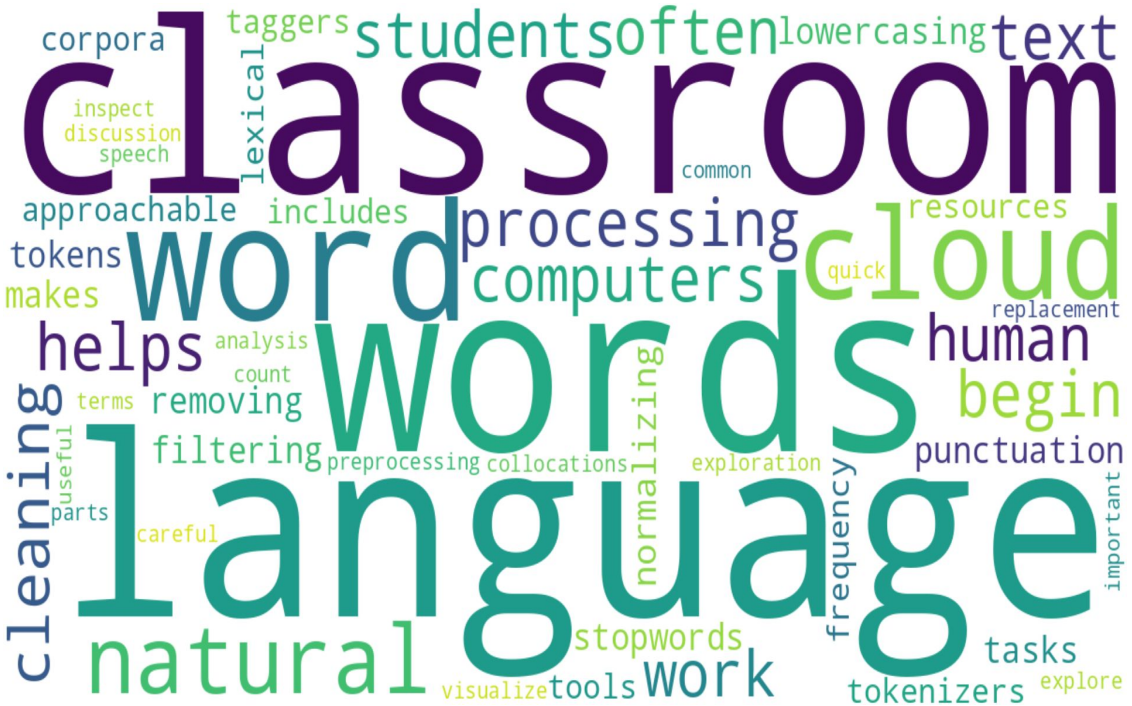


Word cloud

- A popular way is word cloud
- Note that it tells you nothing about the relationship between individual words

```
from wordcloud import WordCloud

cloud = WordCloud(
    width=1200, height=650,
    background_color="white",
    max_words=80, collocations=False,
    random_state=42
).generate_from_frequencies(counts)
```




Time for Python